

Week 7: Representation – Gaussian Networks

Practical Assignment

So far in this module, we have exclusively looked at PGMs with discrete random variables. We now move on to PGMs with continuous random variables, and specifically Gaussian random variables with linear relationships between them.

The assignment consists of two questions:

- The first question is a simple Gaussian Markov network (MNs), which aims to give you a gentle introduction to modelling and inference with Gaussian networks.
- The second question is more complex: It systematically develops the well-known Kalman filter – a dynamic Bayes network (BN) – which is used to track the height of a ball given noisy measurements.

But first, let's look at how to implement Gaussian factors and factor operations in `emdw`.

Information: Implementation of Gaussian factors in `emdw`

Multivariate Gaussian factors in `emdw` is implemented in the `SqrtMVG` class. To use Gaussian factors in `emdw` for today's assignment, you have to use the code discussed below.

Make sure you include these header files – `prlite_genmat.hpp` is necessary for vectors and matrices, and the Gaussian factor is found in `sqrtemvg.hpp`:

```
#include "prlite_genmat.hpp"
#include "sqrtemvg.hpp"
```

The rest of the example code uses the following shorthand:

```
typedef SqrtMVG SG;
```

We will first look at the creation of a Gaussian factor in the moment representation. Suppose we have a two-dimensional Gaussian distribution with the mean vector

$$\boldsymbol{\mu} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (1)$$

This vector is declared using

```
prlite::ColVector<double> mn(2);
```

and the element values assigned using

```
mn[0] = 1; mn[1] = 0;
```

Also suppose the covariance matrix is the identity matrix, or

$$\boldsymbol{\Sigma} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (2)$$

This matrix is created using

```
prlite::RowMatrix<double> cov(2,2);

cov(0,1) = cov(1,0) = 0.0; cov(0,0) = cov(1,1) = 1.0;
```

The Gaussian distribution, $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, can now be created by

```
rcptr<Factor> ptr01(uniqueptr<SG>(new SG({0,1}, mn, cov)));
```

where the identities of the random variables are 0 and 1 respectively.

You can also create the Gaussian factor directly in the canonical representation, $\mathcal{C}(\mathbf{K}, \mathbf{h}, g)$. For numerical stability, the `SqrtMVG` factor does not store the information matrix, $\mathbf{K}$, but rather its square root, $\mathbf{L}$, where $\mathbf{K} = \mathbf{L}\mathbf{L}^T$ and $\mathbf{L}$ is a lower triangular matrix. This is the so-called Cholesky decomposition of the information matrix. If you have defined the square root of the information matrix $\mathbf{L}$, the information vector $\mathbf{h}$ and constant g , you can create the Gaussian factor in `emdw` as follows:

```
rcptr<Factor> ptr01(uniqueptr<SG>(new SG({0,1}, L, h, g)));
```

There are some member functions of the `sqrMVG` class that are not defined in the abstract `Factor` class. To gain access to these member functions when you refer to the Gaussian factor with a pointer to an abstract `Factor` class, you have to downcast the abstract pointer first, which you can do as follows:

```
rcptr<SG> dwnPtr; dwnPtr = dynamic_pointer_cast<SG>(ptr01);
```

You can then access member functions `getMean` (to get the mean vector), `getCov` (to get the covariance matrix), `getK` (to get the information matrix), and `getH` (to get the information vector) as follows:

```
cout << "Mean vector: " << dwnPtr->getMean();
cout << "Covariance matrix: " << dwnPtr->getCov();
cout << "Information matrix: " << dwnPtr->getK();
cout << "Information vector: " << dwnPtr->getH();
```

Another useful member function is `export2DMesh`, which writes a 2-D grid of values to a file; this can then be used for visualisation:

```
dwnPtr->export2DMesh("post_mesh.txt", 0, 1, 128);
```

The first argument specifies the filename, the second and third arguments specify the IDs of the random variables involved, and the fourth argument specifies the number of points per dimension. See `emdw/src/emdw-factors/sqrMVG.hpp` for more information about the arguments and outputs, as well as for more member functions.

The following Python code should create a 3-d mesh plot:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

data = np.loadtxt("post_mesh.txt", dtype=float)
X = np.array(data[0])
Y = np.array(data[1])
X, Y = np.meshgrid(X, Y)
Z = np.array(data[2:])
fig = plt.figure()
ax = fig.add_subplot(projection='3d')
surf = ax.plot_surface(X, Y, Z, cmap=plt.cm.YlGnBu_r)
plt.show()
```

There is also useful functionality that is part of the `SqrtMVG` class that will help you set up a joint Gaussian distribution if the relationship between RVs in the joint distribution is described by an affine (linear) transformation. Suppose that the conditional distribution $p(\mathbf{y}|\mathbf{x})$ is described by the affine (linear) transformation

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b} + \mathbf{n}, \quad (3)$$

where $\mathbf{A}$ is a linear transformation matrix, $\mathbf{b}$ is an offset vector, and $\mathbf{n}$ is a zero-mean noise vector described by

$$\mathbf{n} \sim \mathcal{N}(\mathbf{n}; \mathbf{0}, \Sigma_{\mathbf{n}}). \quad (4)$$

If the distribution $p(\mathbf{x})$ exists as an `SqrtMVG` factor, then the joint distribution $p(\mathbf{x}, \mathbf{y}) = p(\mathbf{y}|\mathbf{x})p(\mathbf{x})$ can be constructed as a pointer to a `Factor` object using

```
rcptr<Factor> ptrXY(
    uniqptr<SG>(
        SG::constructAffineGaussian(*ptrX, A, b, yVars, R)
    )
);
```

where

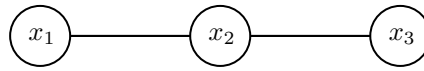
- `ptrX` is a pointer to a `SqrtMVG` factor that represents $p(\mathbf{x})$ ¹
- `A` is a `prlite::ColMatrix<double>` that represents matrix $\mathbf{A}$
- `b` is a `prlite::ColVector<double>` that represents vector $\mathbf{b}$
- `yVars` contain the RV identities corresponding to $\mathbf{y}$; e.g., $\{y1, y2\}$
- `R` is a `prlite::ColMatrix<double>` that represents the *upper triangular* Cholesky decomposition of the noise covariance; i.e., $\mathbf{R}\mathbf{R}^T = \Sigma_{\mathbf{n}}$, where $\mathbf{R}$ is an upper triangular matrix²

¹The assumption here is that the type of `ptrX` is a pointer to `SqrtMVG`; if it is rather a pointer to `Factor`, then you need to downcast the pointer; i.e., replace `*ptrX` by `*dynamic_pointer_cast<SG>(ptrX)`.

²Most Cholesky decomposition routines return the *lower triangular* form. One can find the upper triangular form from such a routine by $\mathbf{R} = [\text{Cholesky}(\Sigma_{\mathbf{n}}^{-1})]^{-T}$. In this module, however, we will mostly use 1-D noise covariance matrices, so the Cholesky decomposition will be trivial to calculate. FYI, the Cholesky decomposition is available in Python as the `cholesky` function available in the `numpy.linalg` library.

1 Modelling and inference for a Gaussian Markov network

Consider the following simple Markov network (MN), where x_1 , x_2 and x_3 are Gaussian random variables:



The factors of this model are given by

$$\phi_1(x_1, x_2) = \mathcal{N}\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}; \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix}\right) \quad (5)$$

and

$$\phi_2(x_2, x_3) = \mathcal{N}\left(\begin{bmatrix} x_2 \\ x_3 \end{bmatrix}; \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \begin{bmatrix} 2 & 2 \\ 2 & 3 \end{bmatrix}\right). \quad (6)$$

- (On paper) Convert these factors to the canonical form and calculate the joint distribution, $p(x_1, x_2, x_3)$, in the canonical form (you do not have to calculate the normalisation constant).
- (In code) Implement these factors in `emdw` and use `emdw`'s factor operators to obtain the joint distribution, $p(x_1, x_2, x_3)$. Extract the information matrix and information vector of the joint distribution and compare it with (a). Also extract the covariance matrix of the joint distribution. Are there any variables that are statistically independent? Are there any variables that are conditionally independent? Can you come to the same conclusions by looking at the MN structure?
- (On paper) Calculate the marginal distribution $p(x_1, x_3)$ in the canonical form³ (you can again ignore the normalisation constant).
- (In code) Use `emdw` to calculate the marginal distribution $p(x_1, x_3)$ and compare the result with that of (c). Also extract the information matrix and covariance matrix – what can you conclude about the (in-)dependence between x_1 and x_3 ? Now export a 2-D mesh of this distribution and plot it (using e.g. Python's `matplotlib`) – what is the relationship between the shape of the distribution and the (in-)dependence of the random variables?
- (On paper) Calculate the conditional distribution $p(x_1, x_3 | x_2 = 1)$ in the canonical form⁴ (you can again ignore the normalisation constant).
- (In code) Use `emdw` to calculate the conditional distribution $p(x_1, x_3 | x_2 = 1)$ and compare the result with that of (e). Also extract the information matrix and covariance matrix – what can you conclude about the conditional (in-)dependence between x_1 and x_3 given x_2 ? Now export a 2-D mesh of this distribution and plot it – what is the relationship between the shape of the distribution and the conditional (in-)dependence of the random variables?
- (In code) Convert the MN to a cluster graph by placing each factor in its own cluster. By only using `emdw`'s factor operators, implement belief propagation and extract the posterior belief $p(x_2, x_3 | x_1 = 0)$ after message passing.
- (In code) Use `emdw`'s built-in functionality to construct the cluster graph, perform belief propagation, and answer the query $p(x_2, x_3 | x_1 = 0)$. Compare the results with that of (g).

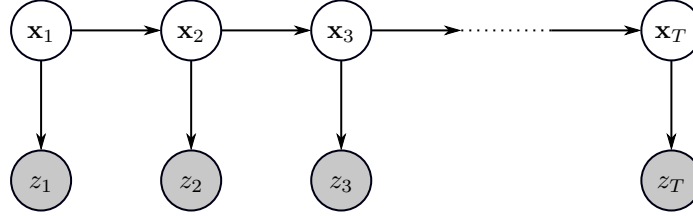
Something to think about: In this question, we have performed inference for a very simple Gaussian network. However, can you see how one would perform inference for a larger Gaussian network (using `emdw`'s built-in functionality)?

³You will have to rearrange the elements in the random vector $\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}^T$ before you can apply the marginalisation operation.

⁴You will again have to rearrange the elements in the random vector $\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}^T$ before you can apply the observe and reduce operation.

2 Tracking a ball with the Kalman filter

The idea of this question is to track the height of a ball thrown into the air (or any other type of projectile, for that matter) given noisy measurements of the ball's height, using the Kalman filter. The Kalman filter uses the dynamic Bayes network shown below.



The Bayes network (BN) is called *dynamic* since it models the behaviour of a system over time. The dynamic BN models T time step, $t \in (1, \dots, T)$, where the length of time between subsequent steps is given by Δt . For each time step t , the system has a number of latent (unobserved) states, $\mathbf{x}_t$, and a number of measurements, z_t . This graph structure should be familiar to you, since it is exactly the same as the one used by hidden Markov models (HMMs) – the only differences between HMMs and Kalman filters are the type of RVs used and the way the relationships between the RVs are described. The dynamic BN used by the Kalman filter assumes that all states and measurements are Gaussian RVs, and that the relationships between RVs are described by linear Gaussian factors.

The problem that the Kalman filter solves is that of *tracking* – it calculates the posterior belief over the states of the *current* time step t given the current and all the previous measurements, or $p(\mathbf{x}_t | z_1, \dots, z_t)$. This means that it does not try to calculate the belief over the states at any of the *past* time steps⁵. In this question, we implement the Kalman filter using the concepts and techniques from the material of this week. Note that this question is only a brief introduction to the Kalman filter – you should study it more thoroughly before you can effectively use it in practice⁶.

Let us now look at the specifics of the model for this question. The latent states at each time step are the height of the ball h_t and the vertical velocity of the ball $\dot{h}_t$ (where up is defined as the positive direction), or $\mathbf{x}_t = [h_t \ \dot{h}_t]^T$. The measurement at each time step, z_t , is a noisy observation of the height of the ball. All the RVs, $\mathbf{x}_t$ and z_t , are assumed to be continuous.

The dynamic BN shown above requires the definition of the factors $p(\mathbf{x}_1)$, $p(\mathbf{x}_t | \mathbf{x}_{t-1})$, and $p(z_t | \mathbf{x}_t)$. The factor $p(\mathbf{x}_t | \mathbf{x}_{t-1})$, often called the *state transition* or *motion model*, is described by the linear Gaussian

$$\mathbf{x}_t = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \mathbf{x}_{t-1} + \begin{bmatrix} -\frac{(\Delta t)^2}{2} \cdot g \\ -\Delta t \cdot g \end{bmatrix} + \mathbf{w}_t, \quad (7)$$

where Δt is the length of time between time steps, $g = 9.81 \text{ m.s}^{-2}$ is the gravitational acceleration constant, and $\mathbf{w}_t$ is additive noise called *process noise*. This linear Gaussian relationship describes how the position and velocity states of an object changes over time in response to gravity, with perturbations (e.g. due to wind) modelled by the process noise. The process noise is modelled as zero-mean, independent Gaussian noise,

$$\mathbf{w}_t \sim \mathcal{N}(\mathbf{w}_t; \mathbf{0}, \Sigma_{\mathbf{w}}), \quad \Sigma_{\mathbf{w}} = \begin{bmatrix} \frac{(\Delta t)^3}{3} \cdot q & \frac{(\Delta t)^2}{2} \cdot q \\ \frac{(\Delta t)^2}{2} \cdot q & \Delta t \cdot q \end{bmatrix}, \quad (8)$$

where q is a constant parameter in the covariance matrix $\Sigma_{\mathbf{w}}$ ⁷.

The factor $p(z_t | \mathbf{x}_t)$, often called the *measurement model*, is given by the linear Gaussian

$$z_t = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathbf{x}_t + v_t, \quad v_t \sim \mathcal{N}(v_t; 0, \sigma_v^2). \quad (9)$$

⁵The so-called *Kalman smoother* calculates the posterior belief over the states at the current and all the past time steps. Although you should easily be able to see how the Kalman smoother works in principle (it sends messages both forward and backward in time, whereas the Kalman filter only sends messages forward in time), we will not use it in this module.

⁶See e.g. **Barber** Chapter 24, **Murphy** Chapter 8, or **Koller&Friedman** Chapter 15.

⁷The linear Gaussian of Equations 7 and 8 have been derived from a continuous-time model with additive white noise. The constant q in the covariance matrix $\Sigma_{\mathbf{w}}$ is the height of the power spectral density of the white noise in the continuous-time system. If you have done continuous-time random processes before, then this might make some sense to you; if not, then you can ignore the meaning of q .

Here, v_t is zero-mean, independent Gaussian noise called the *measurement noise*, with its variance given by σ_v^2 . The measurement is therefore a noisy observation of the height of the ball, with the measurement error described by the measurement noise.

The factor $p(\mathbf{x}_1)$ represents our prior knowledge about the states of the system (i.e., the initial height and velocity of the ball). We are quite uncertain about the initial states, which is represented by our choice of prior belief:

$$p(\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1; \begin{bmatrix} 10 \\ 0 \end{bmatrix}, \begin{bmatrix} 100 & 0 \\ 0 & 1000 \end{bmatrix}\right). \quad (10)$$

For this question, we use the following parameters:

- The number of time steps $T = 9$
- Time step length $\Delta t = 0.5$ s
- Process noise parameter $q = 1$
- Measurement noise variance $\sigma_v^2 = 9$

The measurements are given by $(z_1, \dots, z_9) = (2.228, 10.41, 19.22, 23.46, 23.62, 26.78, 13.73, 10.44, 6.172)$

Due to numerical issues, `emdw` restricts the information matrix $\mathbf{K}$ of a `SqrtMVG` factor to be positive definite (not positive semi-definite). This means that the information matrix must be invertible, and a linear Gaussian factor can therefore not be represented by a `SqrtMVG` factor (since the information matrix of a linear Gaussian is singular/not invertible). For this reason, we cannot use `emdw`'s built-in functionality to construct a cluster graph (CG) and perform message passing when there are linear Gaussian factors in the model. The approach of this question is to systemically develop a solution, showing you how to handle linear Gaussian factors along the way.

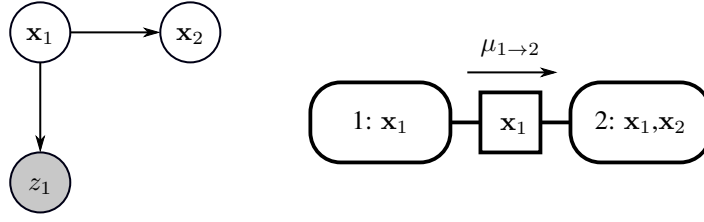
To help you check your work, some intermediate answers are given at the end of this question in blue.

- (a) (On paper) We start with the dynamic BN for only time step 1, shown on the left in the diagram below. Our goal for now is to calculate $p(\mathbf{x}_1|z_1) \propto p(\mathbf{x}_1)p(z_1|\mathbf{x}_1)$ with the evidence $z_1 = 2.228$. The CG we use as framework for inference is shown below on the right; both factors of the model, $p(\mathbf{x}_1)$ and $p(z_1|\mathbf{x}_1)$, are assigned to this single cluster.



Now calculate $p(\mathbf{x}_1|z_1 = 2.228)$ by hand as a canonical factor, using the canonical representation of a linear Gaussian factor applied to the measurement model of Equation 9, as well as Gaussian factor operations, as set out in the notes for this week.

- (b) (In code) Choose a scheme to assign identities to RVs similar to what you have done for the casino HMM in Question 2.2 of Week 4's practical. Construct $p(\mathbf{x}_1)$ as a `SqrtMVG` factor in `emdw`. Now construct $p(\mathbf{x}_1|z_1) \propto p(\mathbf{x}_1)p(z_1|\mathbf{x}_1)$ using the `constructAffineGaussian` function, and use the `observeAndReduce` function to obtain $p(\mathbf{x}_1|z_1 = 2.228)$. Display the information matrix and information vector of $p(\mathbf{x}_1|z_1 = 2.228)$ and compare them with your hand-calculated answer of part (a). Also display the mean vector and covariance matrix of $p(\mathbf{x}_1|z_1 = 2.228)$ – do these make sense to you?
- (c) (On paper) We now add the states at time step 2 as shown on the left in the diagram below. The CG we use as framework for inference is shown on the right below. The goal in this part is to calculate the posterior belief $p(\mathbf{x}_2|z_1)$ with the evidence $z_1 = 2.228$.



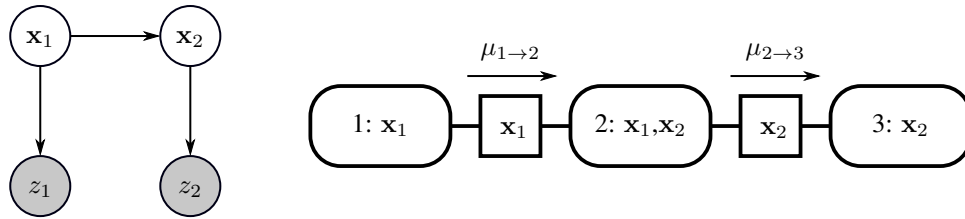
At this point, the numerical hand calculations become very tedious, so we only do *symbolic* manipulations by hand⁸. We assign the new factor, $p(\mathbf{x}_2|\mathbf{x}_1)$, as cluster potential to cluster 2. Symbolically write down how the message $\mu_{1 \rightarrow 2}(\mathbf{x}_1)$ is defined in terms of the quantities in part (a). Then symbolically write down how the cluster belief $\beta_2(\mathbf{x}_1, \mathbf{x}_2) = p(\mathbf{x}_1, \mathbf{x}_2|z_1 = 2.228)$ is defined in terms of $\mu_{1 \rightarrow 2}(\mathbf{x}_1)$ and $p(\mathbf{x}_2|\mathbf{x}_1)$. Lastly, write down how to obtain the marginal belief $p(\mathbf{x}_2|z_1 = 2.228)$.

- (d) (In code) Implement the procedure to calculate $p(\mathbf{x}_2|z_1 = 2.228)$ that you worked out in part (c) in `emd`, using the `constructAffineGaussian` function and factor operations. Note that the Cholesky decomposition of the process noise covariance matrix Σ_w of Equation 8 is given by

$$\Sigma_w = \begin{bmatrix} \frac{(\Delta t)^3}{3} \cdot q & \frac{(\Delta t)^2}{2} \cdot q \\ \frac{(\Delta t)^2}{2} \cdot q & \Delta t \cdot q \end{bmatrix} = \begin{bmatrix} \frac{\Delta t \sqrt{\Delta t \cdot q}}{2\sqrt{3}} & \frac{\Delta t \sqrt{\Delta t \cdot q}}{2} \\ 0 & \sqrt{\Delta t \cdot q} \end{bmatrix} \begin{bmatrix} \frac{\Delta t \sqrt{\Delta t \cdot q}}{2\sqrt{3}} & 0 \\ \frac{\Delta t \sqrt{\Delta t \cdot q}}{2} & \sqrt{\Delta t \cdot q} \end{bmatrix} = \mathbf{R}\mathbf{R}^T, \quad (11)$$

where $\mathbf{R}$ is an upper triangular matrix. Display and inspect the mean vector and covariance matrix of $p(\mathbf{x}_2|z_1 = 2.228)$. Does it make sense to you?

- (e) (On paper) Next we add the measurement at time step 2, z_2 , as shown on the left in the diagram below. The corresponding CG is shown on the right below. The goal of this part is to calculate the posterior belief $p(\mathbf{x}_2|z_1, z_2)$ with the evidence $(z_1, z_2) = (2.228, 10.41)$.



We assign the new factor, $p(z_2 = 10.41|\mathbf{x}_2)$, as cluster potential to cluster 3. Symbolically write down how the message $\mu_{2 \rightarrow 3}(\mathbf{x}_2)$ is defined in terms of the quantities in part (c). Then write down symbolically how to calculate the cluster belief $\beta_3(\mathbf{x}_2) = p(\mathbf{x}_2|z_1 = 2.228, z_2 = 10.41)$ in terms of the message $\mu_{2 \rightarrow 3}(\mathbf{x}_2)$ and cluster potential $p(z_2 = 10.41|\mathbf{x}_2)$.

- (f) (In code) Implement the procedure to calculate $p(\mathbf{x}_2|z_1 = 2.228, z_2 = 10.41)$ that you worked out in part (e) in `emd`, using the `constructAffineGaussian` function and factor operations. Display the mean vector and covariance matrix of $p(\mathbf{x}_2|z_1 = 2.228, z_2 = 10.41)$ and compare it with that of $p(\mathbf{x}_2|z_1 = 2.228)$ that you calculated in part (d) – can you see the effect of adding the measurement $z_2 = 10.41$?
- (g) (In code) You should now be able to see how to extend model further: For each new time step t , you perform the calculations in part (d) to obtain $p(\mathbf{x}_t|z_1, \dots, z_{t-1})$ ⁹, and then you perform the calculations in part (f) to obtain $p(\mathbf{x}_t|z_1, \dots, z_t)$ ¹⁰. Place your calculations of parts (d) and (f) in a loop and use it to track the ball for time steps $t = 2$ to $t = 9$. For each time step, store the mean and standard deviation of the posterior beliefs over both the height h_t and the vertical velocity $\dot{h}_t$.

⁸Numerical hand calculations would be “more of the same” at this point and would probably not contribute much to your understanding of the material. However, if you feel that you do not understand the underlying concepts well after going through the hand calculations of the example in the notes, the homework problem for this week, and the calculations of part (a), then it might be worthwhile repeating the numerical calculations of part (c) by hand.

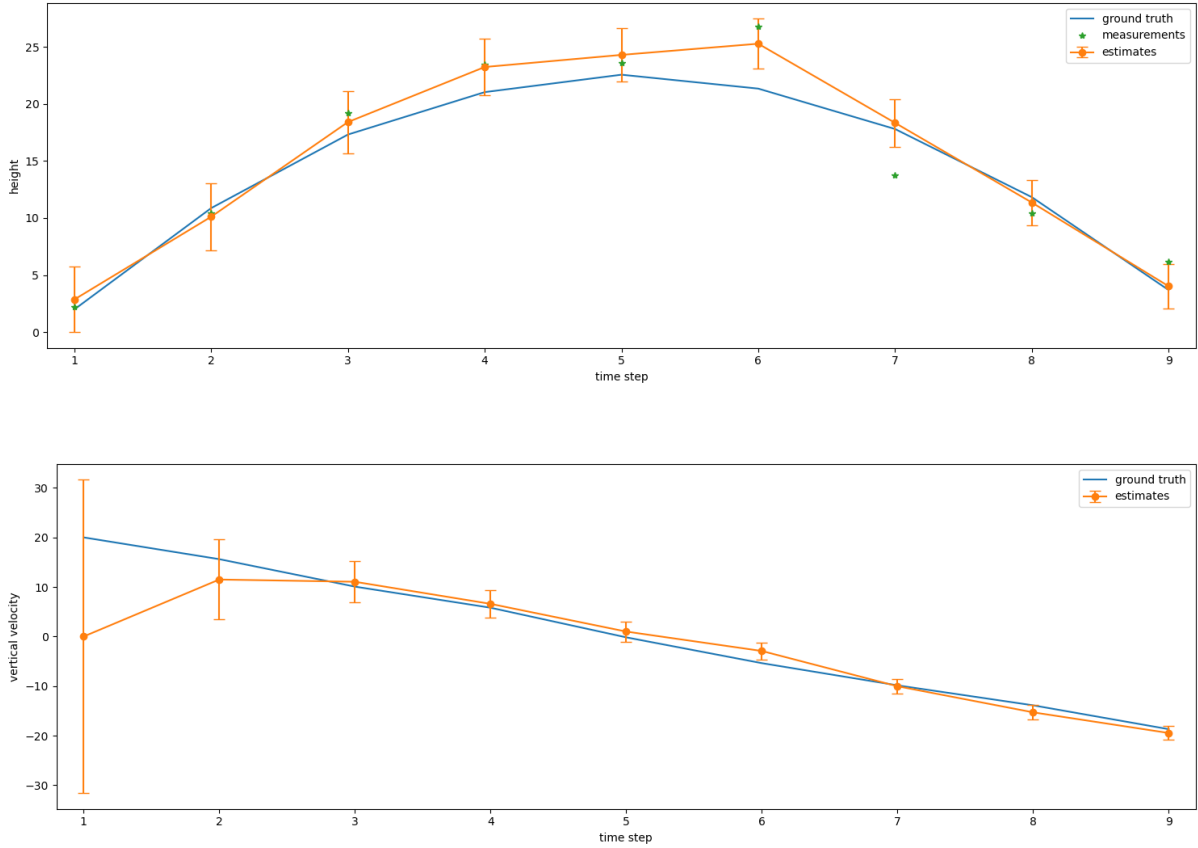
⁹This is called the *prediction step* in the Kalman filter – it predicts the states at the current time step based on the measurements at all the previous time steps.

¹⁰This called the *measurement update step* in the Kalman filter – it updates the prediction with the measurement at the current time step.

(h) (In code) Extract the means and standard deviations of the height h_t and vertical velocity $\dot{h}_t$ ¹¹. Plot the means and standard deviations of these estimates together with the measurements and ground truth (using e.g. Python). The ground truth for the heights and vertical velocities are respectively given by:

- $(h_1, \dots, h_9) = (2.000, 10.87, 17.31, 21.03, 22.56, 21.34, 17.80, 11.83, 3.698)$
- $(\dot{h}_1, \dots, \dot{h}_9) = (20.00, 15.61, 10.07, 5.810, -0.1651, -5.374, -9.854, -13.88, -18.70)$

You should get something like the two plots below.



Closely study the behaviour shown in the plots. Can you explain all the effects?

Answers:

(a) $p(\mathbf{x}_1|z_1 = 2.228) = \mathcal{C}\left(\mathbf{x}_1; \begin{bmatrix} 0.1211 & 0 \\ 0 & 0.001 \end{bmatrix}, \begin{bmatrix} 0.3476 \\ 0 \end{bmatrix}, -\right)$

(b) $p(\mathbf{x}_1|z_1 = 2.228) = \mathcal{N}\left(\mathbf{x}_1; \begin{bmatrix} 2.870 \\ 0 \end{bmatrix}, \begin{bmatrix} 8.257 & 0 \\ 0 & 1000 \end{bmatrix}\right)$

(c)

(d) $p(\mathbf{x}_2|z_1 = 2.228) = \mathcal{N}\left(\mathbf{x}_1; \begin{bmatrix} 1.643 \\ -4.905 \end{bmatrix}, \begin{bmatrix} 258.3 & 500.1 \\ 500.1 & 1001 \end{bmatrix}\right)$

(e)

(f) $p(\mathbf{x}_2|z_1 = 2.228, z_2 = 10.41) = \mathcal{N}\left(\mathbf{x}_1; \begin{bmatrix} 10.11 \\ 11.50 \end{bmatrix}, \begin{bmatrix} 8.697 & 16.84 \\ 16.84 & 64.75 \end{bmatrix}\right)$

(g)

(h)

¹¹For this questions, the least amount of effort might be to display these estimates on the screen, and then copy and paste them into e.g. a Python script.

3 Assessment: Class test

There will be short, written class test at the end of the practical period in the practical venue (see STEMLearn for details). If you have worked through the practical assignment and understood everything, then the test should be easy!